seen tests that fail at a particular time of day (because time zones used inconsistently push part of the code into the next day). I've seen tests that pass in Chicago but fail in California. I've seen tests that fail on the first day of a new month and a new year—and, of course, tests that fail on the boundary in and out of daylight savings time. Most of this can be prevented.

When I was young and got paid to write Java, I solved this problem by adding an optional argument to every method that had a default value of `Date.today`, allowing an optional time to explicitly be passed to the method for testing. This is a lot of work (although, interestingly, I think the Ruby community is coming back to allowing this kind of optional argument injection as a regular practice). Here are a few other options for dealing with date and time data.

Using Relative Dates

Using relative dates in your test data is often a way to work around date and time weirdness. You can do this with fixtures, factories, or just the objects you create in your tests.

You can adjust the test to use dates relative to now:

```
let(:runway) { create(:project, name: "Project Runway",
  start_date: 1.week.ago }
let(:greenlight) { create(:project, name: "Project Greenlight",
  start_date: 1.month.ago }
let(:gutenberg) { create(:project, name: "Project Gutenberg",
  start_date: 1.day.ago }

it "finds recently started projects" do
  actual = Project.find_recently_started(6.months)
  expect(actual.size).to eq(3)
end
```

Since Rails fixture files are evaluated as ERB files before loading, you can specify dynamic dates in fixture files, as well:

```
runway:
  name: Project Runway
  start_date: <%= 1.month.ago %>

greenlight:
  name: Project Greenlight
  start_date: <%= 1.week.ago %>

gutenberg:
  name: Project Gutenberg
  start_date: <%= 1.day.ago %>
```

With fixtures written like this, the previous test will always work since the `start_date` of the projects will never fall out of the six-month range.